RELATÓRIO TÉCNICO – PARTE TEÓRICA

Entrega contínua da plataforma “Pedidos Veloz” em microsserviços: do Docker Compose ao Kubernetes com observabilidade e CI/CD

Disciplina: Cloud DevOps – Orchestrating Containers and Micro Services | Aluno(a): [SEU NOME] | Outubro/2026

1. Contexto

A Loja Veloz sofre com indisponibilidade durante deploys, dificuldade de escalar em picos e baixa rastreabilidade de falhas. Este relatório fundamenta a proposta Cloud DevOps para a aplicação “Pedidos Veloz” (API Gateway, Pedidos, Pagamentos, Estoque e PostgreSQL) e justifica as decisões que a Parte Prática implementa, com MVP previsto para 4 semanas.

2. Microsserviços e o papel do DevOps em ambientes cloud-native

A **arquitetura de microsserviços** divide uma aplicação em serviços pequenos, independentes e organizados por capacidade de negócio, cada um com seu próprio ciclo de deploy e comunicação por APIs bem definidas. Isso permite que Pedidos escale sem escalar Pagamentos e que uma falha em Estoque não derrube o sistema inteiro. O custo é a complexidade operacional: muitos serviços, rede entre eles e falhas parciais difíceis de diagnosticar.

O **DevOps** é o que torna esse modelo viável. Em ambiente *cloud-native* ele une cultura e automação: infraestrutura como código, pipelines que constroem e testam cada mudança, deploys frequentes e de baixo risco, e telemetria para reagir rápido. A metodologia **12-Factor App** guia o desenho dos serviços: configuração por variáveis de ambiente, processos stateless e descartáveis, port binding e logs como fluxo de eventos no stdout.

3. Conteinerização: Docker x Kubernetes

Um **contêiner** empacota a aplicação com suas dependências em uma imagem imutável, que roda igual no notebook do dev, no CI e em produção. Diferente de uma VM, compartilha o kernel do host, o que o torna leve e rápido de iniciar. Imagens enxutas (*multi-stage build*), versionadas e publicadas em um *registry* são a unidade de entrega do pipeline.

Docker (com Docker Compose) constrói imagens e executa contêineres em **um único host**: é ideal para desenvolvimento local, testes de integração e ambientes simples. **Kubernetes** orquestra contêineres em **um cluster de várias máquinas**, com estado desejado declarado, autorrecuperação, balanceamento, escala automática e atualização gradual. Eles são complementares: no projeto, o Compose padroniza o ambiente do desenvolvedor e o Kubernetes executa a produção, ambos usando as mesmas imagens.

4. Fundamentação teórica

4.1 Orquestração de contêineres

O Kubernetes funciona por reconciliação: o operador declara o estado desejado em manifests e os controladores trabalham continuamente para mantê-lo. Os objetos centrais são o **Pod** (menor unidade executável), o **Deployment** (réplicas e estratégia de atualização), o **Service** (endereço estável e balanceamento entre Pods), o **ConfigMap** e o **Secret** (configuração e credenciais fora da imagem). **Probes** de readiness e liveness informam quando um Pod pode receber tráfego ou deve ser reiniciado. O **HPA** ajusta o número de réplicas por métricas (ex.: CPU) e o **VPA** recomenda ou ajusta requests de CPU/memória. A segurança é reforçada pelo **Pod Security Admission** e por NetworkPolicies.

4.2 CI/CD em ambientes distribuídos

Integração Contínua valida cada commit automaticamente (lint, testes, build); **Entrega/Implantação Contínua** publica artefatos versionados e os leva aos ambientes com *gates* de aprovação. Com microsserviços, o pipeline precisa construir e versionar cada serviço de forma independente, validar os manifests, escanear imagens e usar *secrets* do próprio CI, nunca credenciais no código. Estratégias de deploy reduzem risco: **Rolling** troca Pods gradualmente; **Blue-Green** mantém duas versões e alterna o tráfego de uma vez; **Canary** envia um percentual pequeno do tráfego à nova versão antes de liberar para todos.

4.3 Observabilidade: métricas, logs e traces

Observabilidade é a capacidade de entender o estado interno do sistema a partir do que ele emite. **Métricas** são séries numéricas agregadas (latência, taxa de erro, uso de CPU) usadas em dashboards, alertas e no HPA. **Logs** são eventos detalhados; em JSON e com *trace_id* tornam-se pesquisáveis. **Traces** distribuídos

seguem uma requisição por todos os serviços, mostrando onde está a lentidão ou o erro. O **OpenTelemetry** padroniza a coleta dos três sinais e propaga o contexto entre serviços (W3C Trace Context). Um **service mesh** como o Istio pode gerar telemetria e mTLS sem alterar código, ao custo de um sidecar por Pod.

5. Exemplo de referência

O caso principal é o **Online Boutique** (GoogleCloudPlatform/microservices-demo), aplicação de e-commerce mantida pelo Google com 11 microsserviços em linguagens diferentes que se comunicam por gRPC. O repositório demonstra o uso de Kubernetes/GKE, Istio, variações de deploy com Kustomize, provisionamento com Terraform e um gerador de carga com Locust. Complementarmente, o **OpenTelemetry Demo** (Astronomy Shop, projeto da CNCF) mostra uma loja em microsserviços totalmente instrumentada com OpenTelemetry, executável tanto com Docker Compose quanto em Kubernetes. Deles a proposta adota: serviços por domínio de negócio, manifests declarativos, IaC, Collector central de telemetria e o mesmo fluxo Compose → Kubernetes, simplificado para o porte da Loja Veloz (REST em vez de gRPC e sem service mesh no MVP).

6. Decisões arquiteturais adotadas

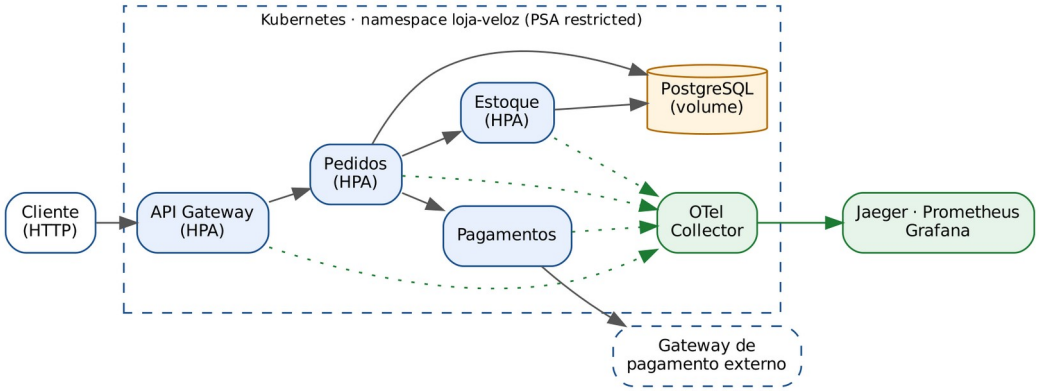


Figura 1 – Arquitetura proposta para a plataforma Pedidos Veloz (elaborada pelo autor).

Decisão	Escolha	Justificativa
Linguagem/serviços	Python + FastAPI, 4 serviços stateless	Imagens pequenas, OpenAPI automático e auto-instrumentação OpenTelemetry madura; estado fica só no PostgreSQL (12-Factor, processos descartáveis).
Comunicação	REST síncrono com timeout e retry; mensageria fora do MVP	Reduz peças móveis no prazo de 4 semanas. O evento “PedidoCriado” (RabbitMQ) fica como evolução, quando houver consumidores assíncronos reais.
Ambiente local	Docker Compose: redes frontend/backend, volume pgdata, arquivo .env	Um único comando (docker compose up) sobe tudo igual para todos os devs; healthchecks garantem ordem de inicialização.
Imagens	Multi-stage, python-slim, usuário não-root, tag semver + SHA, GHCR	Menos superfície de ataque e de tamanho; o SHA torna cada imagem rastreável até o commit e permite rollback exato.
Kubernetes	Deployments, Services, ConfigMap, Secret, probes, requests/limits	Configuração separada do código; readiness evita tráfego para Pod não pronto e liveness reinicia Pod travado.
Segurança	Pod Security Admission “restricted”, securityContext, NetworkPolicy	PSA é nativo (substituiu a PodSecurityPolicy) e bloqueia root, escalonamento de privilégio e capabilities extras.
CI/CD	GitHub Actions: lint → testes → build → scan Trivy → push	Gates impedem que imagem sem testes ou com vulnerabilidade crítica seja publicada; credenciais via GitHub Secrets.
Deploy	Rolling Update (maxUnavailable 0, maxSurge 1)	Nativo, sem custo extra de infra e com zero indisponibilidade quando combinado a readiness probes; canary com Istio é a evolução.

Decisão	Escolha	Justificativa
Escala	HPA por CPU (gateway, pedidos, estoque); VPA só recomendação	Os picos da campanha são de tráfego, logo escala horizontal. VPA em modo Off ajusta requests sem conflitar com o HPA.
Observabilidade	OpenTelemetry → Collector → Jaeger e Prometheus/Grafana; logs JSON	Padrão aberto e independente de fornecedor; trace_id nos logs liga log, métrica e trace da mesma requisição.
IaC	Terraform (esqueleto com módulos e variáveis por ambiente)	Infra reproduzível e versionada no mesmo repositório; state remoto evita divergência entre ambientes.

O serviço de Pagamentos fica com réplicas fixas, pois seu gargalo é o limite de requisições do provedor externo, e não CPU. Em conjunto, essas decisões atacam diretamente os três problemas da empresa: deploys sem indisponibilidade (rolling + probes + gates no CI), escala sob demanda (HPA) e diagnóstico rápido de falhas (traces correlacionados com logs e métricas).

Referências

DOCKER. Docker Docs: Dockerfile, multi-stage builds e Compose. Disponível em: https://docs.docker.com/. Acesso em: out. 2026.

GITHUB. GitHub Actions documentation. Disponível em: https://docs.github.com/actions. Acesso em: out. 2026.

GOOGLE CLOUD. Online Boutique (microservices-demo). Disponível em: https://github.com/GoogleCloudPlatform/microservices-demo. Acesso em: out. 2026.

HASHICORP. Terraform documentation. Disponível em: https://developer.hashicorp.com/terraform/docs. Acesso em: out. 2026.

ISTIO. Istio documentation. Disponível em: https://istio.io/latest/docs/. Acesso em: out. 2026.

KUBERNETES. Kubernetes documentation: Concepts. Disponível em: https://kubernetes.io/docs/concepts/. Acesso em: out. 2026.

OPENTELEMETRY. Documentation e OpenTelemetry Demo. Disponível em: https://opentelemetry.io/docs/ e https://github.com/open-telemetry/opentelemetry-demo. Acesso em: out. 2026.

WIGGINS, A. The Twelve-Factor App. Disponível em: https://12factor.net/. Acesso em: out. 2026.